Programação Concorrente

Comunicação Entre Processos - Parte 01

Professor: **Jeremias Moreira Gomes**
E-mail: jeremias.gomes@idp.edu.br

Introdução

Introdução

- Processos/*threads* quase sempre precisam comunicar-se com outros processos/*threads*
  - Exemplo:
    - Pipeline do Interpretador de Comandos
- É de interesse que essa comunicação ocorra de uma maneira bem estruturada e de preferência sem o uso de interrupções
- Questões relacionadas a essa comunicação são chamadas de Comunicação Entre Processos (*Inter Process Communication* (**IPC**))

Introdução

- A base da comunicação entre processos, reside em três perguntas:
  a. Como um processo pode **passar informações** a outro?
  b. Como processos podem **não atrapalhar** um ao outro?
  c. Qual o **sequenciamento ideal** de processos quando dependências estão presentes?

Introdução

- Processos executando concorrentemente podem ser de dois tipos:
  - **Independentes**
    - Um processo é independente se **não puder afetar** ou ser afetado por **outros processos** em execução no sistema
    - Qualquer processo que não compartilhe dados com qualquer outro processo é independente.

Introdução

- Processos executando concorrentemente podem ser de dois tipos:
  - **Cooperativos**
    - Processo que **pode afetar** ou ser afetado por **outros processos** em execução no sistema.
    - Qualquer processo que compartilhe dados com outros processos.

Introdução

- Existem **dois modelos fundamentais de comunicação** entre processos:
  - **Memória Compartilhada**
    - É estabelecida uma área compartilhada para que os processos possam ler ou escrever nessa área
  - **Memória Distribuída**
    - A comunicação é toda feita por meio de troca de mensagens

IPC

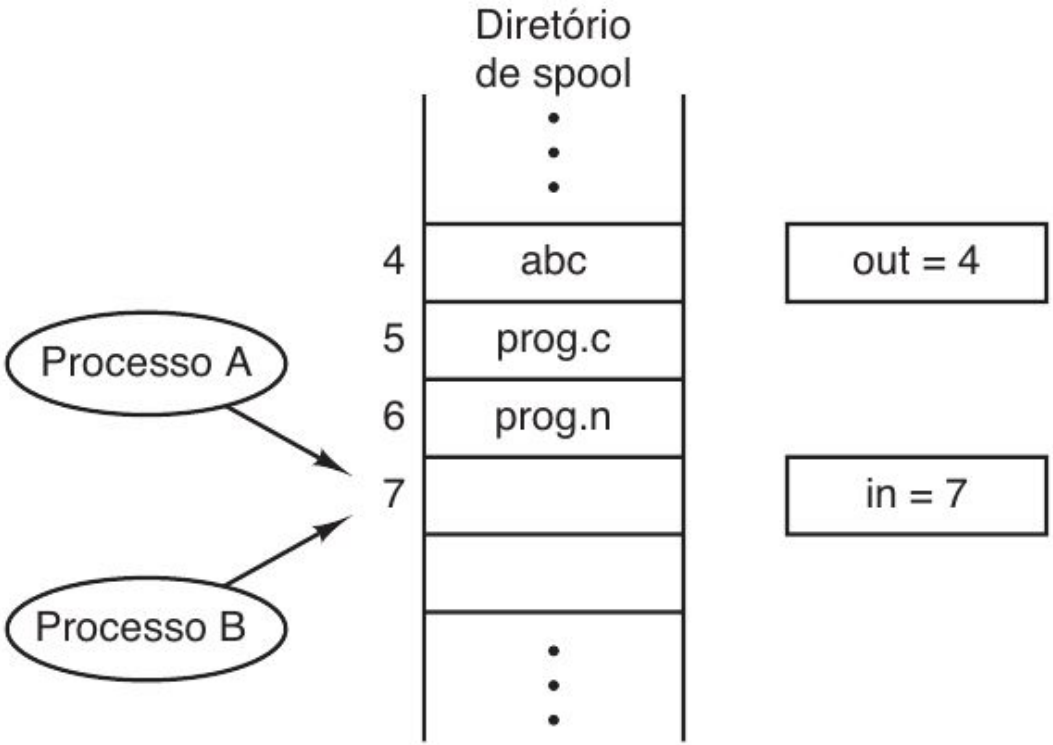
- Diversos mecanismos estão presentes em diversos SOs, para permitir a comunicação entre processos:
  - **Sinais**
  - **Pipes (nomeados ou não)**
  - **Memória compartilhada**
  - Arquivo mapeado de memória
  - Fila de Mensagens (message queue)
  - Passagem de mensagens (message passing)
  - Sockets (de rede ou locais)
  - Arquivos

Condições de Corrida

Condições de Corrida

- São situações onde **dois ou mais processos estão acessando dados compartilhados**
- O resultado final do processamento depende de quem executa e quando executa

Condições de Corrida - Exemplo

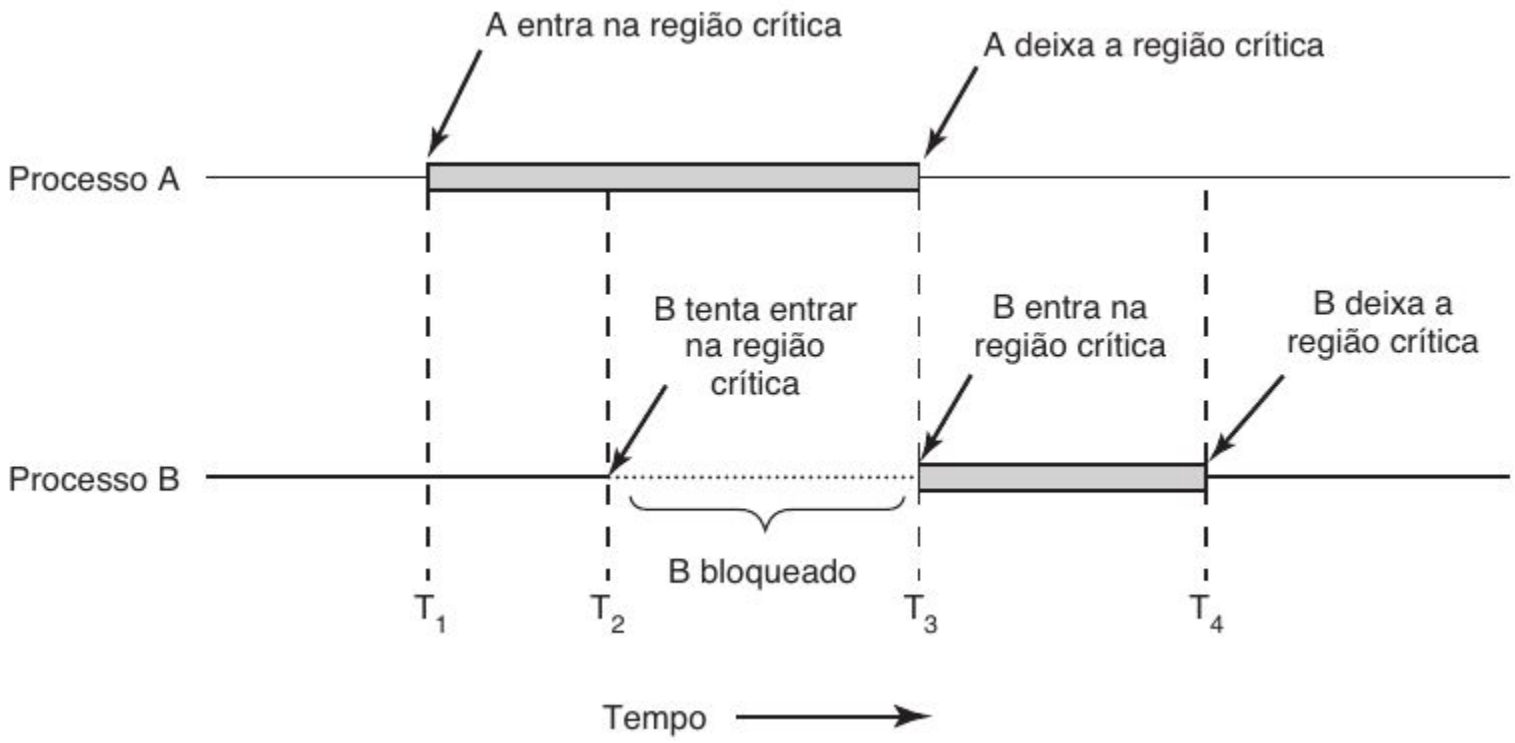


Condições de Corrida - Como Evitar?

- Encontrar uma forma de **proibir que mais de um processo acesse** o dado compartilhado **ao mesmo tempo**
- Desenvolver algo que seja **mutuamente exclusivo** para acesso de um único processo
    - **Exclusão Mútua**: Impedir que dois ou mais processos acessem um mesmo recurso ao mesmo tempo
    - **Região Crítica**: parte do código onde é feito o acesso à memória compartilhada (que pode levar a condições de corrida)

Exclusão Mútua com Dois Processos

Exclusão Mútua - Regiões Críticas



Exclusão Mútua - Regiões Críticas

- **Fluxo de execução** de um processo em uma Região Crítica
  - Executa região **não** crítica
  - Executa procedimento para entrar na região crítica
  - Executa sua região crítica
  - Executa procedimento para deixar a região crítica
  - Executa região **não** crítica

Exclusão Mútua - Região Crítica

- Para evitar condições de corrida, são necessárias a verificação de **quatro condições**:
  a. Dois ou mais processos não podem estar simultaneamente dentro de suas regiões críticas
  b. Nenhuma consideração pode ser feita a respeito da velocidade relativa dos processos, ou a respeito dos processadores disponíveis

Exclusão Mútua - Região Crítica

- Para evitar condições de corrida, são necessárias a verificação de **quatro condições**:

  c. Nenhum processo que esteja fora de sua região crítica pode bloquear a execução de outro processo

  d. Nenhum processo pode ser a obrigado a esperar indefinidamente para entrar em sua região crítica

Soluções (e Tentativas de Soluções) para Exclusão Mútua

Soluções para Exclusão Mútua - Espera Ocupada

- As soluções apresentadas a seguir utilizam o que é chamado de Espera Ocupada (***busy wait***)
- Espera Ocupada é uma técnica em que um processo verifica uma condição repetidamente até que ela seja verdadeira
  - Também é utilizada para gerar atrasos na execução do sistema
  - Possui a desvantagem de **consumir continuamente o processador**

Soluções para Exclusão Mútua - Espera Ocupada

```
int lock = 1;
void *colocarfeijao(void *empty)
{
    while(lock) {};                          // Nunca escrever código assim
    printf("Feijão colocado no prato.\n");
}
void *colocararroz(void *empty)
{
    sleep(10);
    printf("Arroz colocado no prato.\n");
    lock = 0;
}
int main()
{
    pthread_t t1, t2;
    pthread_create(&t1, NULL, colocarfeijao, NULL);
    pthread_create(&t2, NULL, colocararroz, NULL);
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    return 0;
}
```

Soluções para Exclusão Mútua

- A seguir, serão apresentadas algumas soluções para o problema da Exclusão Mútua
  - **Desabilitação de Interrupções**
  - **(Tentativas de) Soluções de Software**
  - **Soluções de Hardware**

Desabilitar Interrupções

- Consiste em **desabilitar as interrupções de cada processo após o seu ingresso na região crítica**.
- As interrupções são habilitadas após o processo sair da região crítica.
- Perigos e Desvantagens
  - **Não é boa prática dar esse poder aos processos**.
  - **Não funciona em computadores com mais de uma CPU**.
    - A desabilitação funciona somente por CPU.

Soluções em Software - *Locks* (Variáveis Tipo Trava)

- É um **mecanismo de sincronização** entre processos/threads de forma que estes sejam programados baseados na verificação de uma variável compartilhada
- Se outro processo solicitar acesso a uma variável que está bloqueada, o processo deverá esperar até que a variável seja destravada

Locks (Variáveis do Tipo Trava)

- Sabendo-se que execuções individuais de threads tem o mesmo efeito correto quando executada sozinha, então pode-se inferir que se as threads forem executadas uma por vez, em alguma ordem, o efeito também será correto.

Locks (Variáveis do Tipo Trava)

- Uma intercalação das operações das *threads* em que o efeito combinado é igual ao que seria se as *threads* tivessem sido executadas uma por vez, em alguma ordem, é uma **intercalação equivalente serialmente**
- O uso de equivalência serial evita a ocorrência de atualizações perdidas ou recuperações inconsistentes

Locks (Variáveis do Tipo Trava)

- Definição:
  - Um lock é **uma variável única compartilhada cujo valor pode assumir valor 0 ou 1**.
  - O valor 0 significa que não há nenhum processo executando a sua região crítica, e o valor em 1 significa que algum processo está executando sua região crítica.

Locks (Variáveis do Tipo Trava)

- Problema:
  - Dois processos A e B tentam entrar em suas regiões críticas simultaneamente
    - A pega o valor do *lock* que é 0
    - A resolve atualizar o valor do *lock* para 1, para poder utilizar a região crítica
    - Antes do *lock* ser atualizado, B acessa o *lock* e vê que também está em 0
    - Esse conflito de verificação é uma Condição de Corrida

Tentativa de Solução para Regiões Críticas 01

```
// variável global: recurso (inicialmente FALSE)
 // Enquanto recurso estiver em uso
 while (recurso == TRUE) {
     ...
}

recurso = TRUE;

// CÓDIGO DA REGIÃO CRÍTICA

recurso = FALSE;

// ...
```

Tentativa de Solução para Regiões Críticas 01

- Essa tentativa não é correta porque processos podem chegar a conclusão simultaneamente que o acesso a Região Crítica está livre
    - Ambos os processos leem recurso como FALSE

Tentativa de Solução para Regiões Críticas 02

- Forma de acesso a região crítica baseada em software.
- Utiliza uma variável chamada turno (ou *turn*).
  - Responsável por estabelecer de quem é a vez na região crítica.

```
// PROCESSO 0
while (1) {
    while (turno != 0) {}
    // REGIÃO CRÍTICA
    turn = 1;
    // REGIÃO NÃO CRÍTICA
}
```

```
// PROCESSO 1
while (1) {
    while (turno != 1) {}
    // REGIÃO CRÍTICA
    turn = 0;
    // REGIÃO NÃO CRÍTICA
}
```

Tentativa de Solução para Regiões Críticas 02

- A principal desvantagem do chaveamento obrigatório é o chaveamento obrigatório.
  - A execução da sessão crítica de um processo sempre vai depender do término da execução do outro processo na sessão crítica (o que pode demorar)
  - A solução requer a entrada estritamente alternada e isso não é bom se um processo é mais lento que o outro.

Tentativa de Solução para Regiões Críticas 03

```
while (quer[outro] == TRUE) {
}
quer[eu] = TRUE

// REGIÃO CRÍTICA

quer[eu] = FALSE; // Após o uso
```

Tentativa de Solução para Regiões Críticas 03

- A solução anterior não assegura exclusão mútua
- Problema similar à tentativa de solução 1
- Os processos podem testar simultaneamente a variável **`quer`** (um testa a do outro) e chegar à conclusão juntos de que a região está liberada

Tentativa de Solução para Regiões Críticas 04

- Uso de intenções de uso da Região Crítica
- Um processo marca sua intenção antes de testar a intenção do outro
- Se ele chega a conclusão que o outro processo não quer entrar na região crítica, é porque ele ainda não testou a intenção do primeiro, portanto, não irão concluir uso juntos

Tentativa de Solução para Regiões Críticas 04

```
// Intenção de Usar a Região Crítica
lock[eu] = TRUE;
while (1) {
    if (lock[outro] == FALSE) {
        break;
    }
}
// REGIÃO CRÍTICA
lock[eu] = FALSE; // Após o uso
```

Tentativa de Solução para Regiões Críticas 04

- Uso de intenções garante exclusão mútua.
- O problema da abordagem é a **chance de ficarem presos em um laço infinito**.
  - Ambos marcam a intenção de entrar simultaneamente.
  - Dessa forma, ambos não saem mais do laço.

Tentativa de Solução para Regiões Críticas 05

- Uso de intenções com verificação
- No laço, agora, o processo verifica se o outro quer entrar na região crítica
  - Caso positivo, ele cede a vez.

Tentativa de Solução para Regiões Críticas 05

```
inicio:
    quer[eu] = TRUE;
    if (quer[outro] == TRUE) {
            lock[eu] = FALSE;
            goto inicio;
        }
    }

    // REGIÃO CRÍTICA

    lock[eu] = FALSE; // Após o uso
```

Tentativa de Solução para Regiões Críticas 05

- Uso de intenções com verificação
  - Garante exclusão mútua
- Processos podem ficar indefinidamente cedendo a vez
  - Raro acontecer, mas o caso existe (Lei de Murphy)

Algoritmo de Dekker

- Primeira proposta de solução completa e funcional
- Similar à proposta da Tentativa 05
- Uso de uma variável chamada vez, para realizar o desempate

Algoritmo de Dekker

```
// PROCESSO 0
quer[0] = TRUE;
while(quer[1] == TRUE) {
    if (turno != 0) {
        quer[0] = FALSE;
        while(turno != 0) { }
        quer[0] = TRUE;
    }
}
// REGIÃO CRÍTICA

turno = 1;
quer[0] = FALSE;
```

```
// PROCESSO 1
quer[1] = TRUE;
while(quer[0] == TRUE) {
    if (turno != 1) {
        quer[1] != FALSE;
        while(turno != 1) { }
        quer[1] = TRUE;
    }
}
// REGIÃO CRÍTICA

turno = 0;
quer[1] = FALSE;
```

Algoritmo de Peterson

- Melhoria para o Algoritmo de Dekker
  - 20 anos depois (1981)
- Reduz o número de *loops*
- Aproveita a própria condição de corrida para decidir o acesso a região crítica
- O controle é realizado em duas rotinas: **`enter_region()`** e **`leave_region()`**

Algoritmo de Peterson

```
// variáveis globais: vez e interessados[2] = {FALSE, FALSE}

enter_region(int processo)
{
    int outro;
    outro = 1 - processo; // 0 ou 1 (só faz uma inversão)
    interessados[processo] = TRUE;
    vez = processo;
    while (vez == processo && interessados[outro] == TRUE) {};
}

leave_region(int processo)
{
    interessado[processo] = FALSE;
}
```

Algoritmo de Peterson

```
// ...

enter_region(p);
// Região Crítica
leave_region(p);

// ...
```

Algoritmo de Peterson

- Características:
  - Preserva a exclusão mútua
  - Permite o progresso
  - Satisfaz a espera limitada
  - Resolve o problema somente para 2 processos

Soluções em Hardware

- Além das implementações em softwares citadas, soluções comerciais possuem instruções de hardware que auxiliam na implementação de exclusão mútua
  - TSL (*Test and Set Lock*) é uma instrução que testa e modifica um valor de maneira atômica (indivisível)
    - Lê o conteúdo da memória, a palavra *lock*, no registrador RX e armazena um valor diferente de zero no espaço de memória *lock*
  - Quando termina de executar a região crítica, muda o valor para zero

Soluções em Hardware

- TSL (*Test and Set Lock*) - Implementação:

```
while (test&set(v) != 0) {}
// Seção Crítica
reset(v);
```